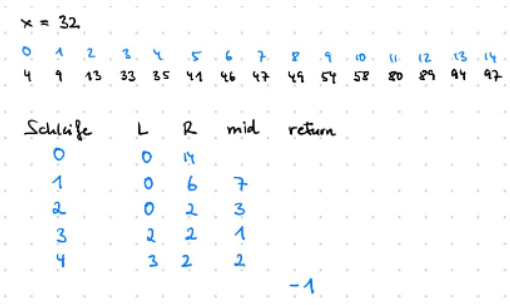
1. (4 Punkte) Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid und den Rückgabewert wenn die binäre Suche auf die Liste a und das gesuchte Element x angewendet wird.

```
def binaereSuche(a, x):
    L = 0
    R = len(a)-1
    while L <= R:
        mid = (L+R)//2
        if a[mid] == x:
            return mid
        if a[mid] < x:
            L = mid + 1
        else:
            R = mid - 1
    return -1

x = 32
a = [4, 9, 13, 33, 35, 41, 46, 47, 49, 54, 58, 80, 89, 94, 97]
```

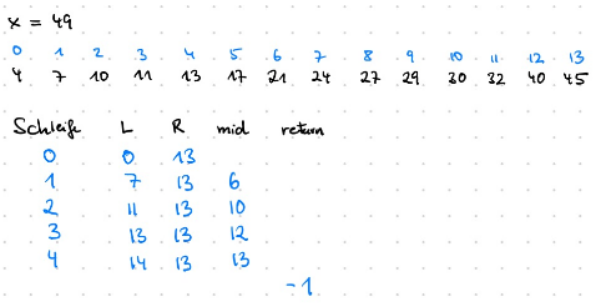
Lösung:



2. (4 Punkte) Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid und den Rückgabewert wenn die binäre Suche auf die Liste a und das gesuchte Element x angewendet wird.

```
x = 49
a = [4, 7, 10, 11, 13, 17, 21, 24, 27, 29, 30, 32, 40, 45]
```

Lösung:



3. (4 Punkte) Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid und den Rückgabewert wenn die binäre Suche auf die Liste a und das gesuchte Element x angewendet wird.

```
x = 13
a = [4, 7, 10, 11, 13, 17, 21, 24]
```

Lösung:

x = 13

	0	1	2	3	4	5	6	7
	4	7	10	11	13	17	21	24

Schleife	L	R	mid	return
0	0	7		
1	4	7	3	
2	4	4	5	
3	4	4	4	4

4. (3 Punkte) (a) (1 Punkt) Eine sortierte Liste hat 1000, 1 Million bzw. 1 Milliarde Elemente. Wie viele Schleifendurchläufe braucht die binäre Suche höchstens? Begründe kurz.

   (b) (1 Punkt) Wie viele Elemente muss die lineare Suche bei einer Milliarde Elementen höchstens ansehen?

   (c) (1 Punkt) Was liefert die binäre Suche aus dem Unterricht für die Liste `[5, 1, 4, 2, 3]` und `x = 1`? Warum?

Lösung:

a) 10, 20 bzw. 30. Jeder Durchlauf halbiert den Suchbereich, und $1000 \approx 2^{10}$, 1 Million $\approx 2^{20}$, 1 Milliarde $\approx 2^{30}$.

b) 1 Milliarde, wenn `x` am Ende steht oder nicht in der Liste ist.

c) `-1`, obwohl 1 in der Liste steht: `mid = 2`, `a[2] = 4 > 1`, also wird links weitergesucht; `mid = 0`, `a[0] = 5 > 1`, dann ist `R = -1`. Die Liste ist nicht sortiert, die binäre Suche funktioniert nur für sortierte Listen.

5. (4 Punkte) Die Funktion `anzahl_kleiner` gibt für eine sortierte Liste `a` zurück, wie viele Elemente kleiner als `x` sind.

```
def anzahl_kleiner(a, x):
    L = 0
    R = len(a)-1
    ans = -1
    while L <= R:
        mid = L + (R-L)//2
        if a[mid] < x:
            ans = mid
            L = mid + 1
        else:
            R = mid - 1
    return ans + 1

a = [3, 5, 5, 5, 8, 12, 15]
```

(a) (3 Punkte) Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid, ans und den Rückgabewert für `anzahl_kleiner(a, 5)`.

(b) (1 Punkt) Wie kann man mit `anzahl_kleiner` berechnen, wie oft eine ganze Zahl `x` in `a` vorkommt?

Lösung:

a)

Schleife	L	R	mid	ans	return
0	0	6		−1	
1	0	2	3	−1	
2	0	0	1	−1	
3	1	0	0	0	
					1

b) `anzahl_kleiner(a, x+1) - anzahl_kleiner(a, x)`, z.B. $4 - 1 = 3$ für `x = 5`.

6. (4 Punkte) Die Liste a war ursprünglich sortiert. Dann wurde ein Anfangsteil nach hinten verschoben. Die Funktion rotatedList sucht mit binärer Suche den Index des alten Anfangs. Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid, ans und den Rückgabewert.

```
def rotatedList(a):
    L = 0
    R = len(a)-1
    ans = -1
    while L <= R:
        mid = (L+R)//2
        if a[0] <= a[mid] :
            ans = mid
            L = mid + 1
        else:
            R = mid - 1
    if ans == len(a)-1:
        return 0
    return ans+1

a = [21, 24, 27, 29, 30, 32, 40, 45, 4, 7, 10, 11, 13, 17]
```

Lösung:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
	21	24	27	29	30	32	40	45	4	7	10	11	13	17

Schleife	L	R	mid	ans	return
0	0	13		-1	
1	7	13	6	6	
2	7	9	10	6	
3	7	7	8	6	
4	8	7	7	7	
5					8

7. (4 Punkte) Die Werte der Liste a sind erst aufsteigend, dann absteigend. Die Funktion findMaximum sucht mit binärer Suche den Index des größten Elements. Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid, ans und den Rückgabewert.

```
def findMaximum(a):
    L = 0
    R = len(a)-1
    ans = -1
    while L <= R:
        mid = (L+R)//2
        if mid == 0 or a[mid] > a[mid-1]:
            ans = mid
            L = mid + 1
        else:
            R = mid - 1
    return ans

a = [21, 24, 27, 44, 6, 2]
```

Lösung:

0	1	2	3	4	5
21	24	27	44	6	2

Schleife	L	R	mid	ans	return
0	0	5		-1	
1	3	5	2	2	
2	3	3	4	2	
3	4	3	3	3	
					3

8. (3 Punkte) Eine frühere Version von `findMaximum` sah so aus:

```
def findMaximum(a):
    L = 0
    R = len(a)-1
    ans = -1
    while L <= R:
        mid = (L+R)//2
        if a[mid] > a[mid-1]:
            ans = mid
            L = mid + 1
        else:
            R = mid - 1
    return ans

a = [1, 9, 8, 7, 6]
```

(a) (2 Punkte) Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für L, R, mid, ans und den Rückgabewert.

(b) (1 Punkt) Erkläre, warum das Ergebnis falsch ist, und korrigiere die Funktion.

Lösung:

a)

Schleife	L	R	mid	ans	return
0	0	4		−1	
1	0	1	2	−1	
2	0	−1	0	−1	
					−1

b) Richtig wäre 1. Bei `mid = 0` wird `a[0]` mit `a[-1]` verglichen, also mit dem letzten Element 6. Weil $1 < 6$ ist, wird links weitergesucht. Korrektur: `if mid == 0 or a[mid] > a[mid-1]:`

9. (4 Punkte) Schreibe eine Funktion `wurzel(n)`, die mit binärer Suche die größte ganze Zahl `k` mit $k \cdot k \leq n$ bestimmt. Durchsuche dazu die Zahlen von 0 bis `n`.

```
'''
>>> wurzel(16)
4
>>> wurzel(99)
9
>>> wurzel(0)
0
'''
```

Tipp: Ordne jeder Zahl `k` den Wert True zu, wenn $k \cdot k \leq n$ ist.

Lösung:

```
def wurzel(n):
    L = 0
    R = n
    ans = -1
    while L <= R:
        mid = L + (R-L)//2
        if mid * mid <= n:
            ans = mid              # mid passt, rechts nach einer größeren Zahl suchen
            L = mid + 1
        else:
            R = mid - 1
    return ans
```